2020年7月20日 10:03

- 前往->移动安全管理平台
- 移动安全产品
- 1.1. 安卓应用加固系统
  - 1.1.1. 云平台在线使用
  - 1.1.2. 云平台API接口
  - 1.1.3. 本地私有部署
  - 1.1.4. Java2C白名单设置
- 1.2. 源码混淆系统
  - 1.2.1. 获取工具
  - 1.2.2. iOS版
  - 1.2.3. Android版
  - 1.2.4. 常见问题
- 1.3. 源码虚拟化系统
  - 1.3.1. 获取工具
  - 1.3.2. iOS版
  - 1.3.3. Android版
- 1.4. 密钥白盒SDK
  - 1.4.1. 获取工具
  - 1.4.2. iOS版
  - 1.4.3. Android版
  - 1.4.4. 常见问题
- 其他安全产品
- 2.1. H5源码加密系统
  - 2.1.1. 云平台在线使用
  - 2.1.2. 云平台API接口
- 2.2. Lua源码加密系统
  - 2.2.1. Cocos手游加密
  - 2.2.2. xLua项目加密
- Published with GitBook

Android版

Android版使用说明

1.概述

本文主要介绍Android版本的安全密钥白盒如何使用，阅读者需要一定的Android的开发经验，否则可能存在一定的疑问

2.拷贝.so文件

在Android Stuido中打开项目，将解压文件夹下armeabi-v7a 和x86 中的.so 文件到项目jniLibs对应的文件夹中，若没有jniLibs文件请自行创建，如下图

3.拷贝.aar文件

将解压包中的KiwiWBSDK.aar文件到项目的libs文件中，如下图：

4.配置build.gradle文件

4.1 在Android Studio中打开build.gradle文件，在android节点下的defaultConfig中 配置 ndk引用armeabi-v7a 和 x86文件的路径，在repositories中配置libs；以及在dependencies节点下配置 .aar 文件的引用，配置代码如下图： 配置代码如下

```
defaultConfig {
    xxxx
    ndk {
        abiFilters 'armeabi','x86'
```

```
    }
}
repositories{
    flatDir{
        dirs 'libs'
    }
}
dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar'])
    implementation (name:'KiwiWBSKD-x.x.x', ext:'aar')
}
```

4.2 配置截图如下：

5.导入SDK包

在项目中添加下面的代码，导入密钥白盒SDK包

```
import com.kiwisec.whitebox.KiwiWBSDK;
```

6.调用加解密函数

6.1 函数描述：

- 调用SDK提供的加密、解密算法，无需提供密钥key信息，无需担心密钥泄露问题。

6.2 加密函数：

- public static byte[] encrypt(int keyType, String input)
- public static byte[] encrypt(int keyType, byte[] input)

6.3 解密函数：

- public static byte[] decrypt(int keyType, byte[] input)

6.4 参数说明

- input：待加密数据，String或byte数据类型
- keyType：由算法类型和算法模式经过或运用所得，不支持算命模式则为算法类型

6.5 支持算法类型

- AES、BLOWFISH、CAMELLIA、CAST128、CAST256、DES、DESX、GOST、MISTY1、RC4、RC5、SAFERK64、SEED、SKIPJACK、TDES、TWOFISH、XTEA、XXTEA、SM4、BASE64

6.6 支持算法模式

- ECB、CBC、CFB、OFB、CTR，其中SM4暂仅支持ECB和CBC模式，RC4、RC5为序列算法和BASE64不支持算法模式

6.7 返回数据

- 加密或解密后的数据

6.8 代码实例

- AES算法使用代码如下：

```
KiwiWBSDK.encrypt(KiwiWBSDK.CRYPT_AES, input)//默认为ECB模式
KiwiWBSDK.encrypt(KiwiWBSDK.CRYPT_AES|KiwiWBSDK.CRYPT_CBC, input)
KiwiWBSDK.encrypt(KiwiWBSDK.CRYPT_AES|KiwiWBSDK.CRYPT_CFB, input)
KiwiWBSDK.encrypt(KiwiWBSDK.CRYPT_AES|KiwiWBSDK.CRYPT_OFB, input)
KiwiWBSDK.encrypt(KiwiWBSDK.CRYPT_AES|KiwiWBSDK.CRYPT_CTR, input)
```

6.9 白盒密钥偏移量IV说明

- 在CBC,OFB,CFB,CTR,PCPB加密模式下，必须提供偏移量IV，白盒密钥的偏移IV会根据Key(密钥)与加密的BlockSize（块大小）生成。
- 生成原理
- 第一种情况：Key长度大于BlockSize【目前自动生成的密钥都是这种情况】，从Key中截取长度为BlockSize长度的字符并倒序处理 例如：iv = Key.subString(0,BlockSize);//iv=g04SCvsa4dYxdGjumode_iv = ujGdxYd4asvCS40g
- 第二种情况：Key长度小于BlockSize【自定义的密钥可能存在】，使用PKCS5Padding方式进行填充，填充BlockSize - Key.length个字符，并倒序处理

7.调用加签函数

7.1 函数描述

- 网络数据在传输过程中存在被篡改或伪装的攻击，通过对数据进行加签操作，便于数据接收端检验数据的有效性

7.2 数据加签函数

```
public static byte[] sign(int signType, byte[] input)
public static byte[] sign(int signType, String input)
```

7.3 参数说明

- Input：待加签的数据，String或byte数据类型
- signType摘要算法：MD5、SHA_224、SHA_256、SHA_384、SHA_512、HMAC_SHA224、HMAC_SHA256、HMAC_SHA384、HMAC_SHA512、SM3、HMAC_SM3

7.4 返回数据

- 加签后的摘要数据

7.5 代码实例

- 例如MD5摘要计算代码如下：

```
KiwiWBSDK.sign(KiwiWBSDK.CRYPT_MD5, input)
```

8.调用验签函数

8.1 函数描述

- 网络数据在传输过程中存在被篡改或伪装的攻击，通过对接收到的数据进行验签操作，确认数据的有效性
- 数据验签函数：

```
public static boolean verify(int signType, String input, String signInfo)
public static boolean verify(int signType, byte[] input, String signInfo)
```

8.2 参数说明

- Input：待验签的数据，String或byte数据类型
- signType验签算法：MD5、SHA_224、SHA_256、SHA_384、SHA_512、HMAC_SHA224、HMAC_SHA256、HMAC_SHA384、HMAC_SHA512、SM3、HMAC_SM3

8.3 返回数据

- 验签后的摘要数据

8.4 代码实例

- 例如MD5验签计算代码如下：

```
KiwiWBSDK.verify(KiwiWBSDK.CRYPT_MD5, inputString, sigInfo)
```

9.联系我们

如遇问题请联系我们的客服或技术支持，公司电话：028-83177117

来自 <https://doc.kiwisec.com/kiwiSdk/android-sdk.html>